

ML-Based Pre-Screening and Verification Referral for Lithography Hotspots on ICCAD-12 Benchmarks

Bonthala Sai Kiran
(24BVD1106)

Banda
Sirichandana
(24BVD1114)

Kamal Sharma
(24BVD1118)

Karthikeyan R V
(24BVD1119)

Abstract—Lithography hotspot detection is important because layout patterns that are difficult to print can lead to opens, shorts, and yield loss and commonly treated as a binary classification problem in which layout patterns are classified as hotspot (HS) or non-hotspot (NHS). A Conventional lithographic binary classifier, however, forces every prediction into one of two decisions even when the model is uncertain, while detailed verification can be computationally expensive. This work implements a three-stage confidence-based pre-screening and verification-referral flow on five ICCAD-12 benchmarks. Stage 1 trains a lightweight 64×64 grayscale convolutional neural network (CNN) and produces a hotspot confidence score for every test pattern. Stage 2 converts that score into three routing decisions—safe, risky, and needs check—using three threshold pairs: 0.30/0.70, 0.20/0.80, and 0.10/0.90. Stage 3 compares the proposed routing against the baseline 0.50 cutoff and measures unchecked-hotspot rate, screening rate, flagged rate, and referral rate. At the featured 0.20/0.80 setting, the macro-average unchecked-hotspot rate decreases from 11.94% for the baseline to 4.77%, a reduction of 7.17 percentage points, while 92.43% of patterns are screened out and 3.44% are referred for detailed verification on a macro-average basis. The threshold study shows the expected workload–risk trade-off: 0.10/0.90 reduces unchecked hotspots further to 1.99% but raises the referral rate to 5.68%. The results demonstrate the value of adding a decision layer, while the final verification accuracy, calibration, runtime, and actual simulation-cost savings remain to be measured.

Keywords—Lithography hotspot detection, ICCAD-12, Convolutional Neural Network, Confidence score, Pre-screening, Verification referral.

I. INTRODUCTION

Lithography hotspots are layout patterns that may print incorrectly during manufacturing, producing defects such as opens or shorts. The project addresses a practical screening problem: detailed verification is valuable but expensive, so a fast ML stage where a trained model can learn visual characteristics of hotspot and non-hotspot layout patterns and produce a rapid prediction that can be used to prioritize which patterns deserve further attention. The supplied project brief frames the research question as whether confidence-based referral can reduce the number of real hotspots that are incorrectly screened out while reducing verification workload.

The key contribution of this implementation is not a new CNN architecture. Instead, the CNN is used as a first-level scanner that outputs a continuous hotspot confidence score. A decision layer then

uses confidence thresholds to route patterns to safe, risky, needs check automatically. This changes the system from a forced binary decision into a selective pipeline that can abstain from uncertain cases and refer them to detailed verification.

II. LITERATURE REVIEW

The reference paper supplied studies lithography hotspot detection on the ICCAD-12 benchmark under severe class imbalance. The paper compares transfer learning using VGG16 features with classical classifiers and a lightweight custom CNN trained from scratch. The custom CNN contains 12,873 parameters, reports an average validation balanced accuracy of 95.3%, and reports an average inference time of 6.3 ms per classification on its stated evaluation setup. The present implementation uses a separate lightweight CNN and evaluates it on saved test predictions. These reference results are therefore contextual rather than directly comparable performance numbers because the models and evaluation protocols differ.

The implemented extension introduces a confidence-based routing layer. The same test confidence scores are evaluated under a 0.50 binary baseline and under all three proposed threshold pairs. This controlled comparison isolates the effect of the decision layer from changes in the underlying test predictions.

III. DATASET ANALYSIS

The project uses five ICCAD-12 benchmark directories, each containing training and testing images separated into hotspot (HS) and non-hotspot (NHS) classes. The supplied test predictions contain 146,277 patterns in total, including 2,750 hotspots (1.88%). Each benchmark uses a separate training and test folder; 15% of the training folder is used as validation data. The strong class imbalance makes accuracy alone insufficient, so precision, recall/sensitivity, specificity, F1-score, and balanced accuracy are reported for the baseline.

TABLE I
TEST-SET COMPOSITION ACROSS THE FIVE ICCAD-12
BENCHMARKS.

Benchmark	HS	NHS	Test patterns	HS share
B1	226	4,679	4,905	4.61%
B2	498	41,298	41,796	1.19%
B3	1,808	46,333	48,141	3.76%
B4	177	31,890	32,067	0.55%
B5	41	19,327	19,368	0.21%
Total	2,750	143,527	146,277	1.88%

IV. SYSTEM ARCHITECTURE

The complete system has three stages. Stage 1 predicts a hotspot confidence score. Stage 2 interprets that score using thresholds and routes each pattern. Stage 3 quantifies whether the routing layer reduces unchecked hotspots and how much verification workload is potentially avoided.

- Stage 1 — PREDICT: layout image $\rightarrow$ preprocessing $\rightarrow$ CNN $\rightarrow$ hotspot confidence $C \in [0,1] \rightarrow$ baseline metrics.
- Stage 2 — DECIDE: confidence $C \rightarrow$ SAFE / NEEDS_CHECK / RISKY using lower and upper thresholds.
- Stage 3 — PROVE: baseline outputs + referral outputs $\rightarrow$ comparison tables, confusion analysis, and trade-off plots.

The implementation flow is therefore: ICCAD-12 layout $\rightarrow$ 64 $\times$ 64 grayscale CNN $\rightarrow$ hotspot confidence $\rightarrow$ confidence-based screening/referral $\rightarrow$ baseline-versus-proposed analysis.

V. CNN BASELINE AND TRAINING

The compact CNN model is implemented with input of layout’s image represented in grayscale and of 64 $\times$ 64 pixels. Pixel values are normalized to [0,1]. The CNN contains two convolutional blocks followed by a compact classifier: The first convolutional block contains a 16-filter 3 $\times$ 3 ReLU convolution, batch normalization, and 2 $\times$ 2 max pooling. The second block contains a 32-filter 3 $\times$ 3 ReLU convolution, batch normalization, and 2 $\times$ 2 max pooling; The resulting feature map is flattened, passed through 30% dropout and a 64-unit ReLU dense layer, and finally a one-unit sigmoid layer. The model is trained with Adam and binary cross-entropy for 10 epochs with batch size 32. Class weighting compensates for unequal hotspot/non-hotspot frequencies developing an hotspot sensitive architecture.

The sigmoid output is converted to a consistent hotspot confidence so that a larger value always means greater hotspot confidence

TABLE II
IMPLEMENTED CNN ARCHITECTURE.

Block	Layer	Configuration
Input	Input	64 $\times$ 64 $\times$ 1 grayscale
Feature 1	Conv + BN + Pool	16 filters, 3 $\times$ 3, ReLU; 2 $\times$ 2 max pool
Feature 2	Conv + BN + Pool	32 filters, 3 $\times$ 3, ReLU; 2 $\times$ 2 max pool
Classifier	Flatten + Dropout + Dense	Dropout 0.30; Dense 64, ReLU
Output	Dense	1 unit, sigmoid

VI. CONFIDENCE-BASED REFERRAL

For the baseline, the decision threshold is fixed at $C=0.50$: $C \geq 0.50$ is classified as hotspot and $C < 0.50$ as non-hotspot. Unlike the baseline, the proposed system does not force every confidence score into HS or NHS. The second stage takes the per-image hotspot confidence and assigns one of three decisions. For a low threshold L and high threshold H, patterns are classified based on confidence as

$$\begin{aligned} \text{Safe: } C &\leq L \\ \text{Needs Check: } L &< C < H \\ \text{Risky: } C &\geq H \end{aligned}$$

Safe patterns are screened out as likely non-hotspots without detailed verification. Risky patterns are directly flagged as likely hotspots. Needs Check patterns enter the detailed human verification as classifier is not confident enough for an automatic decision. Three threshold pairs are tested: 0.30/0.70, 0.20/0.80, and 0.10/0.90.

The three-way decision is important when interpreting the results. A hotspot placed in safe is an unchecked hotspot in this study. A hotspot placed in needs_check is not counted as a final detection success or

failure because the downstream verifier has not yet been run. Thus, the reported proposed missed-hotspot rate measures only the hotspots that the decision layer would pass without further attention.

TABLE III
MACRO-AVERAGE ROUTING TRADE-OFF ACROSS FIVE BENCHMARKS.

L/H	Screened safe	Flagged	Referred	HS unchecked
0.10/0.90	90.66%	3.66%	5.68%	1.99%
0.20/0.80	92.43%	4.13%	3.44%	4.77%
0.30/0.70	93.40%	4.55%	2.05%	8.01%

VII. RESULTS

A. Baseline Metrics

TABLE IV
BASELINE TEST METRICS SUMMARY.

Bench.	Bal. acc.	Precision	Recall	Specificity	F1
B1	75.02	45.63	53.10	96.94	49.08
B2	98.74	38.49	99.40	98.08	55.49
B3	96.53	43.85	97.95	95.11	60.58
B4	96.62	12.08	97.18	96.07	21.49
B5	93.59	3.45	92.68	94.51	6.66
Macro avg.	92.10	28.70	88.06	96.14	38.66

The baseline CNN performs differently across the five benchmarks. It achieves a high overall recall of 88.06%, indicating that most actual hotspots are detected. However, the low overall precision of 28.70% shows that many patterns predicted as hotspots are actually non-hotspots. B1 has the highest number of missed hotspots, with 106 false negatives, while B4 and B5 show particularly low precision due to a large number of false-positive predictions. The overall specificity of 96.14% indicates that the model is generally effective at identifying non-hotspot patterns.

TABLE V
BASELINE BINARY CONFUSION COUNTS.

Benchmark	TP	FP	FN	TN
B1	120	143	106	4,536
B2	495	791	3	40,507
B3	1,771	2,268	37	44,065
B4	172	1,252	5	30,638
B5	38	1,062	3	18,265

These results show that although the baseline CNN can detect hotspots, its performance is inconsistent across benchmarks, motivating the use of a confidence-based referral mechanism for uncertain predictions.

B. Baseline versus Proposed Unchecked Hotspots

At the featured 0.20/0.80 setting, the macro-average unchecked-hotspot rate falls from 11.94% under the 0.50 baseline to 4.77% under the referral layer, an absolute reduction of 7.17 percentage points. This is a routing improvement: fewer real hotspots are placed in the safe bucket and therefore passed without further attention.

The benchmark-level reduction is not uniform. At 0.20/0.80, the missed-hotspot rate changes from 46.90% to 16.81% for B1, 0.60% to 0.40% for B2, 2.05% to 1.94% for B3, 2.82% to 2.26% for B4, and

7.32% to 2.44% for B5. B1 shows the largest routing change, while B3 changes only slightly because most of its hotspots are already assigned to the risky bucket.

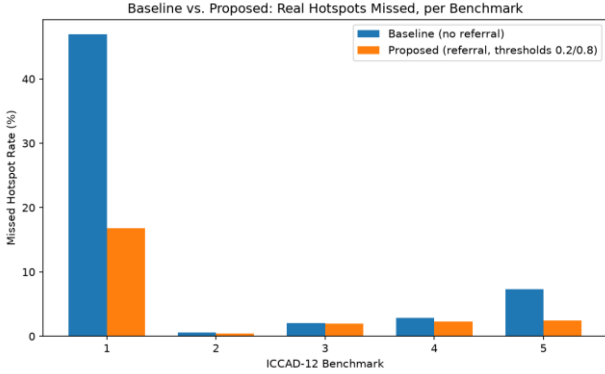


Fig. 1. Baseline versus proposed unchecked-hotspot rate at the featured 0.20/0.80 thresholds.

When all 146,277 test patterns are pooled, the baseline contains 154 false negatives among 2750 real hotspots. At 0.20/0.80, 80 of the 2750 real hotspots are screened as safe. This is a reduction of 74 unchecked hotspots, or 48.1% relative to the baseline false-negative count. The pooled count view and the macro-average rate view are both reported because macro averaging gives every benchmark equal weight, whereas pooled counts are weighted by the number of test patterns.

C. Screening Efficiency and Verification Workload

The proposed routing layer separates the test population into three workload categories. At 0.20/0.80, the macro-average routing shares are 92.43% screened as safe, 4.13% flagged as risky, and 3.44% referred for detailed human verification. If verification would otherwise be applied to every test pattern, referring only the needs_check bucket corresponds to a potential 96.56% reduction in the number of patterns entering that verification queue. This is a potential workload reduction, not a measured runtime or cost saving, because the project has not yet executed the downstream verifier.

The class breakdown is also important. At 0.20/0.80, the pooled needs_check queue contains 2,217 patterns, of which 147 are true hotspots and 2,070 are true non-hotspots. Thus, the current CNN plus referral layer substantially reduces the number of patterns that require detailed verification (around ~92% reduction), but the queue still contains many NHS cases. Reducing unnecessary NHS referrals is therefore a future optimization target.

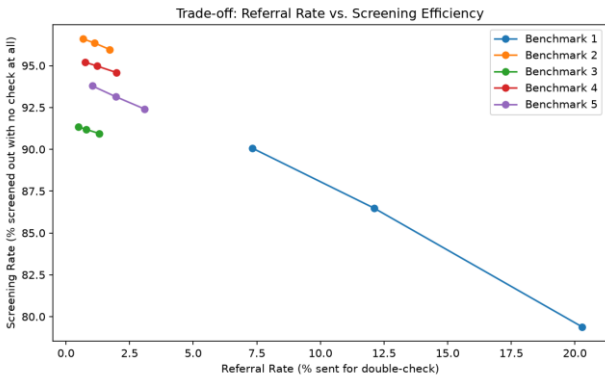


Fig. 2. Referral rate versus screening efficiency across benchmarks and threshold settings.

D. Confusion Matrix Analysis

The baseline naturally produces a 2×2 confusion matrix because it makes a binary HS/NHS decision. The proposed system is

fundamentally a 2×3 routing problem (true HS/NHS versus safe/needs_check/risky), so the primary proposed analysis retains all three routing outcomes.

TABLE VI
PROPOSED 2×3 ROUTING CONFUSION TABLE AT 0.20/0.80 (POOLED).

True class	Safe	Needs_check	Risky	Total
HS	80	147	2,523	2,750
NHS	136,828	2,070	4,629	143,527

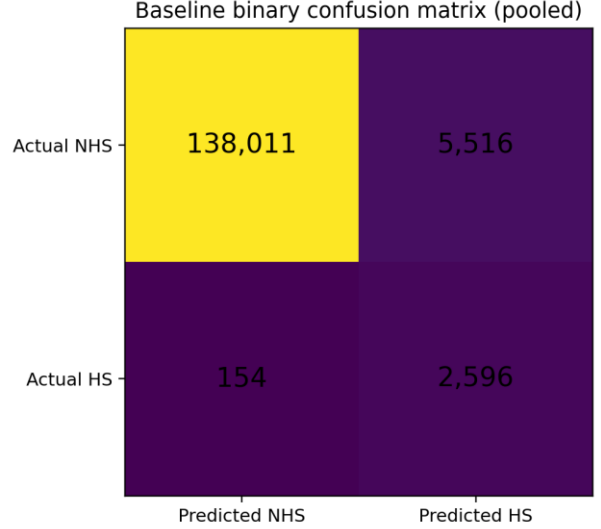


Fig. 3. Pooled binary confusion matrix for the 0.50 baseline.

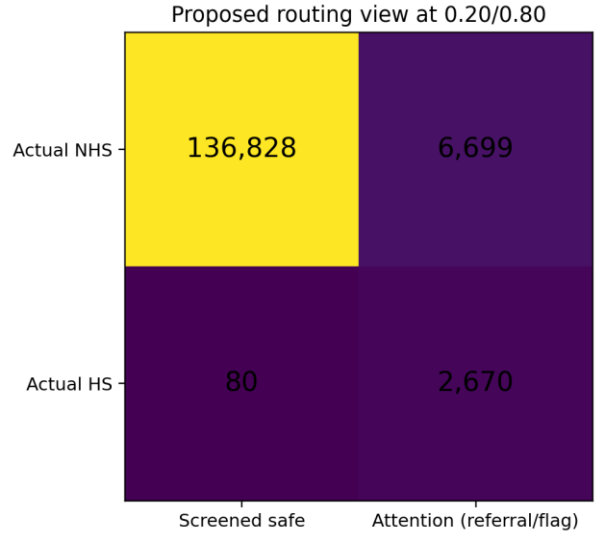


Fig. 4. Pooled proposed routing view at 0.20/0.80. “Attention” combines needs_check and risky only for a safety-oriented 2×2 comparison; it is not a final verification result.

In the pooled safety-oriented 2×2 view, the baseline has 154 real hotspots in the false-negative/pass-as-NHS position, whereas the proposed routing has 80 real hotspots in the screened-safe position. The dangerous unchecked case therefore decreases by 74 patterns. However, this does not prove that the proposed model has a higher final detection accuracy, because the needs_check cases still require a verifier.

E. Threshold Ablation and Tradeoff

Thresholds control how much uncertainty is tolerated. The 0.30/0.70 pair has the narrowest needs_check interval among the tested settings and therefore produces the lowest referral rate, 2.05%, but leaves 8.01% of real hotspots unchecked on a macro-average basis. Moving to 0.20/0.80 increases referral to 3.44% and lowers unchecked hotspots to 4.77%. The 0.10/0.90 pair increases referral further to 5.68% and lowers unchecked hotspots to 1.99%. Thus, sending more uncertain cases for verification trades additional verification workload for fewer hotspots passed unchecked.

TABLE VII
THRESHOLD ABLATION - TRADEOFF.

Low / high	Screened as safe	Flagged	Referred	HS passed unchecked
0.30 / 0.70	93.40%	4.55%	2.05%	8.01%
0.20 / 0.80	92.43%	4.13%	3.44%	4.77%
0.10 / 0.90	90.66%	3.66%	5.68%	1.99%

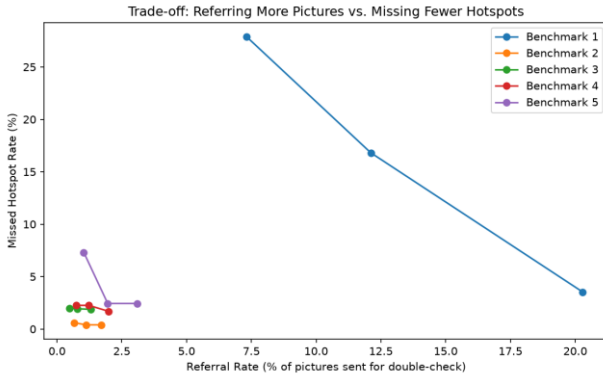


Fig. 5. Referral-rate versus unchecked-hotspot-rate trade-off.

The 0.20/0.80 pair is used as the featured comparison because it occupies an intermediate point between the lower-workload 0.30/0.70 setting and the lower-unchecked-hotspot 0.10/0.90 setting. The experiments do not establish it as a mathematically optimal threshold; threshold selection should instead be calibrated on validation data and tied to an explicit verification-cost objective.

VIII. DISCUSSION

The results show two different kinds of improvement. First, the decision layer changes the routing of uncertain predictions and reduces the fraction of real hotspots that are automatically screened out. Second, it allows most patterns to bypass detailed verification. These gains are complementary: a useful screening system must reduce risk without sending every pattern to the expensive verifier.

The threshold study demonstrates the central engineering trade-off of the project. A wider Needs Check region moves more samples away from automatic decisions and can reduce the number of true hotspots silently accepted as Safe. A narrower referral (Needs Check) region reduces review workload but leaves more cases to automatic classification.

The most important limitation is that the proposed system's needs_check cases do not have downstream ground-truth verification outcomes in the current experiment. Therefore, the 4.77% figure at 0.20/0.80 is an unchecked-hotspot rate, not a final false-negative rate after verification. Likewise, the 92.43% screening rate is a routing statistic, not a measured percentage reduction in wall-clock runtime.

IX. LIMITATIONS AND FUTURE WORK

- Increase input resolution and model capacity. The current 64×64 input is deliberately lightweight, but downsampling can remove fine-grained geometric information from layout patterns. A standard/deeper CNN or a higher-resolution input can be evaluated to determine whether classification accuracy and hotspot recall improve.
- Add an actual downstream verifier. The next stage should run lithography simulation, rule-based verification, or another trusted checker on needs_check patterns and report final detection metrics.
- Measure runtime and cost. Record CNN inference time, verifier time, and end-to-end time to convert the potential workload reduction into measured savings.
- Calibrate confidence scores and select thresholds using validation data rather than choosing thresholds only from the test set.
- Repeat experiments across multiple random seeds and report mean and variance.
- Investigate persistent confident errors, particularly cases where widening the referral band produces only small changes in unchecked hotspots, indicating that model capacity or representation—not thresholding alone—is limiting performance.
- Study stronger architectures, data augmentation, higher-resolution inputs, and potentially transfer-learning or attention-based approaches while retaining the same referral evaluation protocol.

X. CONCLUSION

This work implements a three-stage hotspot pre-screening pipeline in which a lightweight CNN generates confidence scores, a confidence-based decision layer routes patterns to safe/risky/needs_check categories, and a final analysis stage quantifies risk-workload trade-offs. Against the 0.50 baseline, the featured 0.20/0.80 setting reduces the macro-average unchecked-hotspot rate from 11.94% to 4.77% - a 7.17-point reduction, while referring to 3.44% of patterns for detailed verification and screening out 92.43% on a macro-average basis. The threshold ablation further shows that 0.10/0.90 reduces unchecked hotspots to 1.99% at the cost of a 5.68% referral rate. The evidence supports confidence-based pre-screening as a useful decision-layer extension, while final reliability and actual runtime savings require downstream verification experiments.

REFERENCES

- [1] A. Verma, K. Akhilesh Rao, and D. S. Hegde, "Lithography Hotspot Detection using Deep Learning," IIT Bombay, supplied reference paper.
- [2] Digital Assignment II, "AI and Machine Learning for IC," BEVD402L, Group G19 project statement and allocation, September 2026.
- [3] Group G19 implementation: 01_baseline_model.py, 02_referral_logic.py, and 03_final_comparison.py; saved benchmark predictions, referral labels, metrics, and plots.
- [4] D. Ding, X. Wu, J. Ghosh, and D. Z. Pan, "Machine learning based lithographic hotspot detection with critical-feature extraction and classification," in *Proc. IEEE Int. Conf. IC Design and Technology (ICIDT)*, 2009, pp. 1–4, doi: 10.1109/ICIDT.2009.5166300.
- [5] D. Ding, J. A. Torres, C. H. Lin, J. R. Cho, and D. Z. Pan, "High performance lithography hotspot detection with successively refined pattern identifications and machine learning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 30, no. 11, pp. 1621–1634, Nov. 2011, doi: 10.1109/TCAD.2011.2164537.
- [6] H. Yang, Y. Lin, B. Yu, and E. F. Y. Young, "Lithography hotspot detection: From shallow to deep learning," in *Proc. IEEE Int.*

System-on-Chip Conf. (SOCC), 2017, pp. 233–238, doi: 10.1109/SOCC.2017.8226047.

[7] J. W. Park, A. Torres, and X. Song, “Litho-aware machine learning for hotspot detection,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 7, pp. 1510–1514, Jul. 2018, doi: 10.1109/TCAD.2017.2750068.

[8] H. Yang, J. Su, Y. Zou, Y. Ma, B. Yu, and E. F. Y. Young, “Layout hotspot detection with feature tensor generation and deep biased learning,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 6, pp. 1175–1187, Jun. 2019, doi: 10.1109/TCAD.2018.2837078.

[9] X. Zhang, S. Chen, Z. Shao, Y. Niu, and L. Fan, “Enhanced lithographic hotspot detection via multi-task deep learning with synthetic pattern generation,” *IEEE Open J. Comput. Soc.*, vol. 6, pp. 141–152, 2025, doi: 10.1109/OJCS.2024.3510555.

WORK ALLOCATION TABLE

Team Member	Work Done
Bonthala Sai Kiran (24BVD1106)	Prepared and organized the ICCAD-12 dataset, performed image preprocessing, developed and trained the baseline CNN model, and generated the initial hotspot prediction results.
Banda Sirichandana (24BVD1114)	Reviewed related research papers, studied existing lithography hotspot detection methods, formulated the problem and motivation, prepared the literature review, and contributed to the final project documentation.
Kamal Sharma (24BVD1118)	Developed the confidence-based referral mechanism using SAFE, NEEDS_CHECK, and RISKY categories, implemented different confidence thresholds, and generated the referral results.
Karthikeyan R V (24BVD1119)	Conducted the final testing and comparison, analyzed the baseline and proposed results, generated performance tables and charts, and prepared the final result analysis and report documentation.

GITHUB REPOSITORY LINK

<https://github.com/karthi2k06-alt/ML-Based-Pre-Screening-and-Verification-Referral-for-Lithography-Hotspots-on-ICCAD-12-Benchmarks>